Report 4: Groupy - A group membership service

Rui Zhang

October 1, 2025

1 Introduction

This report covers the implementation of Group Membership Service (GMS), which is a coordination form required by distributed systems to handle concurrent events, node crashes, and new node additions. This concept was demonstrated by implementing GMS with atomic multicast, which should be able to handle multiple application layer processes with coordinated states. The status of each node is visualized through any color. When a node wishes to change its state, it must multicast that state to all other nodes in the group. When all nodes in the group synchronize, they will be able to display the same color. The main theme of this work is to gain a deeper understanding of the issues arising from coordination and how to address them.

2 Main problems and solutions

gms1.erl --- Basic Group Member Services

Its function is to implement basic group member management, leaders coordinate all message broadcasts, support nodes to join groups, and not handle failures.

gms2.erl --- Add fault detection

Its new features are: using Erlang: monitor to monitor leaders, triggering elections when a leader crash is detected, automatically electing a new leader (the first node in the Slaves list), and adding a random crash testing mechanism.

gms3.erl --- Reliable message transmission

Its new features are: message sequence number: each message has a unique sequence number, saving the last message: each node records the last received message, retransmission mechanism: new leader retransmits the last message to prevent message

loss, deduplication: automatically discards duplicate messages.

worker.erl --- Application layer process

Its function is to simulate actual distributed applications, maintain a color state {R, G, B}, randomly change colors and synchronize them to all nodes through multicast, and verify whether the states of all nodes are consistent.

Its workflow is to create and connect to GMS group processes, obtain initial states from existing nodes, periodically randomly change colors and broadcast, receive color changes from other nodes and synchronize.

gms1:

```
2> W1 = worker:start(w1, gms1, 1000, 10).
<0.87.0>
worker w1: started as leader with color {0,0,0}
gms1 w1: leader started
worker w1: received state {0,1,0}
3> timer:sleep(200).
ok
4> W2 = worker:start(w2, W1, gms1, 1000, 10).
gms1 w1: adding node, new view: [<0.87.0>,<0.91.0>]
<0.91.0>
gms1 w2: joined group, leader is <0.89.0>
worker w2: received initial view [<0.87.0>,<0.91.0>]
worker w1: received state {0,1,0}
worker w2: started with color {0,1,0}
worker w1: received state {0,5,0}
worker w2: received state {0,5,0}
5> timer:sleep(200).
ok
worker w1: received state {0,5,0}
worker w2: received state {0,5,0}
6> W3 = worker:start(w3, W1, gms1, 1000, 10).
gms1 w1: adding node, new view: [<0.87.0>,<0.91.0>,<0.95.0>]
<0.95.0>
gms1 w2: new view [<0.87.0>,<0.91.0>,<0.95.0>]
gms1 w3: joined group, leader is <0.89.0>
worker w3: received initial view [<0.87.0>,<0.91.0>,<0.95.0>]
worker w1: received state {0,5,0}
worker w2: received state {0,5,0}
worker w3: started with color {0,5,0}
worker w1: received state {0,5,0}
```

gms2:

```
2> W1 = worker:start(w1, gms2, 1000, 10).
<0.87.0>
worker w1: started as leader with color {0,0,0}
3> timer:sleep(200).
ok
4> W2 = worker:start(w2, W1, gms2, 1000, 10).
worker w2: received initial view [<0.87.0>,<0.91.0>]
<0.91.0>
worker w1: received state {0,0,0}
worker w2: started with color {0,0,0}
worker w1: received state {2,0,1}
worker w2: received state {2,0,1}
worker w1: received state {6,0,6}
worker w2: received state {6,0,6}
worker w1: received state {2,5,11}
worker w2: received state {2,5,11}
5> timer:sleep(200).
ok
worker w1: received state {1,4,11}
worker w2: received state {1,4,11}
6> W3 = worker:start(w3, W1, gms2, 1000, 10).
worker w3: received initial view [<0.87.0>,<0.91.0>,<0.95.0>]
<0.95.0>
leader w1: crash
worker w2: received state {1,4,11}
worker w3: started with color {1,4,11}
worker w2: received state {0,4,15}
worker w3: received state {0,4,15}
worker w2: received state {0,8,14}
worker w3: received state {0,8,14}
```

gms3:

```
2> W1 = worker:start(w1, gms3, 1000, 10).
<0.87.0>
worker w1: started as leader with color {0,0,0}
3> timer:sleep(200).
ok
4> W2 = worker:start(w2, W1, gms3, 1000, 10).
worker w2: received initial view [<0.87.0>,<0.91.0>]
<0.91.0>
worker w1: received state {0,0,0}
worker w2: started with color {0,0,0}
worker w1: received state {3,0,0}
worker w2: received state {3,0,0}
worker w1: received state {7,0,0}
worker w2: received state {7,0,0}
worker w1: received state {5,0,5}
worker w2: received state {5,0,5}
5> timer:sleep(200).
ok
worker w1: received state {8,1,6}
worker w2: received state {8,1,6}
worker w1: received state {9,0,8}
worker w2: received state {9,0,8}
6> W3 = worker:start(w3, W1, gms3, 1000, 10).
worker w3: received initial view [<0.87.0>,<0.91.0>,<0.95.0>]
<0.95.0>
worker w1: received state {9,0,8}
worker w2: received state {9,0,8}
worker w3: started with color {9,0,8}
worker w1: received state {9,0,8}
worker w2: received state {9,0,8}
worker w3: received state {9,0,8}
```

```
worker w1: received state {11,8,6}
worker w2: received state {11,8,6}
worker w3: received state {11,8,6}
worker w1: received state {16,12,2}
worker w2: received state {16,12,2}
worker w3: received state {16,12,2}
leader w1: crash
worker w2: received state {20,12,0}
worker w3: received state {20,12,0}
worker w2: received state {25,13,2}
worker w3: received state {25,13,2}
worker w2: received state {28,9,0}
worker w3: received state {28,9,0}
worker w2: received state {29,7,2}
```

```
worker w3: received state {46,29,31}
worker w2: received state {44,34,33}
worker w3: received state {44,34,33}
worker w2: received state {48,38,29}
worker w3: received state {48,38,29}
leader w2: crash
worker w3: received state {47,42,33}
worker w3: received state {45,46,30}
worker w3: received state {48,45,34}
worker w3: received state {53,41,30}
```

Bonus:

gms4.erl

gms4 is an enhanced version based on gms3, which achieves truly reliable message transmission through the ACK confirmation mechanism.

```
3> W1 = worker:start(w1, gms4, 1000, 10).
worker w1: started as leader with color {0,0,0}
gms4 w1: leader started
<0.92.0>
4> timer:sleep(200).
ok
worker w1: received state {0,0,0}
worker w1: received state {0,0,0}
5> W2 = worker:start(w2, W1, gms4, 1000, 10).
gms4 w1: adding node, new view: [<0.92.0>,<0.96.0>]
<0.96.0>
gms4 w2: joined group, leader is <0.93.0>
worker w2: received initial view [<0.92.0>,<0.96.0>]
worker w2: started with color {0,0,0}
worker w1: received state {0,0,0}
worker w2: received state {5,0,0}
worker w1: received state {5,0,0}
worker w2: received state {7,5,0}
worker w1: received state {7,5,0}
6> timer:sleep(200).
ok
worker w2: received state {10,8,3}
worker w1: received state {10,8,3}
worker w2: received state {15,13,4}
worker w1: received state {15,13,4}
7> W3 = worker:start(w3, W1, gms4, 1000, 10).
gms4 w1: adding node, new view: [<0.92.0>,<0.96.0>,<0.100.0>]
<0.100.0>
gms4 w2: new view [<0.92.0>,<0.96.0>,<0.100.0>]
gms4 w3: joined group, leader is <0.93.0>
worker w3: received initial view [<0.92.0>,<0.96.0>,<0.100.0>]
worker w3: started with color {15,13,4}
worker w1: received state {15,13,4}
worker w2: received state {15,13,4}
worker w3: received state {15,13,4}
worker w1: received state {15,13,4}
worker w2: received state {15,13,4}
worker w2: received state {11,12,0}
```

The above results indicate that all requirements have been successfully completed.

3 Evaluation

The evaluation of this work is that in GMS1, all nodes will change their states accordingly, but it cannot cope in the event of a leader node crash. A fault handling mechanism has been introduced in GMS2, which enters an election state to select a new leader when a leader node crash is detected by the slave node. However, gms2 cannot continue after the crash. Therefore, reliable multicast has been implemented in GMS3. The disadvantage of GMS3 is the lack of reliable message transmission. GMS4 has resolved this issue through a

confirmation mechanism. Gms4 will print lost messages, but the nodes will still remain synchronized because the leading node will resend these messages. Otherwise, if there is no confirmation message, the node will lose synchronization. The disadvantage of using confirmation messages is that it can affect performance as the number of messages increases. If the network is congested, the monitoring program may mistakenly believe that the leader node has died, but in fact it is still alive, which can lead to the selection of a new leader node, resulting in the emergence of additional leader nodes.

4 Conclusions

This assignment is not difficult, almost all the code when using gms1-2 is provided in the manual, so it is easy to get started and run. However, gms3 and gms4 require some thinking, but not as much as the first two assignments. The main learning content is to understand the importance of node synchronization and the possible situations that may occur during crashes. I also learned how Erlang passes messages in FIFO order and how messages are not always delivered correctly.